Apartment Hunting

AlgoExpert · Very Hard · Arrays

The problem

- Blocks list which buildings (reqs) they contain.
- Choose the block minimising the farthest distance you'd walk to reach any required building.
- Precompute nearest distance per req with a left and right sweep.

Intuition

- For each requirement, precompute the distance from every block to its nearest building, with a left and a right sweep.

Approach

- 1. For each requirement, sweep left to right and right to left
- 2. Store min distances
- 3. For each block, take the max over requirements
- 4. Pick the block with the smallest max

Java solution

```java
public class ApartmentHunting {
    public static int apartmentHunting(List<Map<String, Boolean>> blocks, String[] reqs) {
        int n = blocks.size();
        int[] worst = new int[n];
        for (String r : reqs) {
            int[] d = new int[n];
            int last = -1_000_000;
            for (int i = 0; i < n; i++) { if (blocks.get(i).getOrDefault(r, false)) last = i; d[i] = i - last; }
            last = 2_000_000;
            for (int i = n - 1; i >= 0; i--) { if (blocks.get(i).getOrDefault(r, false)) last = i; d[i] = Math.min(d[i], last - i); }
            for (int i = 0; i < n; i++) worst[i] = Math.max(worst[i], d[i]);
        }
        int best = 0;
        for (int i = 1; i < n; i++) if (worst[i] < worst[best]) best = i;
        return best;
    }
}
```

Complexity

- Time: O(b*r)
- Space: O(b*r)

Recap

- For each requirement, precompute the distance from every block to its nearest building, with a left and a right sweep.
- Complexity: O(b*r)